

# Proyecto Final de Bases de Datos: Amazon Reviews

Informe técnico detallado del diseño, implementación, visualización y análisis en entorno híbrido

Jorge Ois de Pascual

Miguel Pajuelo Gomez

Abril de 2026

## Índice

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                               | <b>3</b>  |
| 1.1. Objetivos técnicos . . . . .                                    | 3         |
| 1.2. Ficheros entregados . . . . .                                   | 4         |
| 1.3. División del trabajo . . . . .                                  | 4         |
| <b>2. Planteamiento del problema y decisiones de arquitectura</b>    | <b>4</b>  |
| 2.1. Por qué una arquitectura mixta . . . . .                        | 4         |
| 2.2. Criterio de modelado . . . . .                                  | 5         |
| <b>3. Diseño del esquema relacional</b>                              | <b>5</b>  |
| 3.1. Entidades y relaciones . . . . .                                | 5         |
| 3.2. Justificación detallada . . . . .                               | 6         |
| 3.3. Representación de la parte no estructurada en MongoDB . . . . . | 6         |
| <b>4. Implementación de la carga de datos</b>                        | <b>7</b>  |
| 4.1. Archivo de configuración . . . . .                              | 7         |
| 4.2. Conexión y creación de la infraestructura . . . . .             | 7         |
| 4.3. Creación de tablas . . . . .                                    | 8         |
| 4.4. Carga incremental línea a línea . . . . .                       | 9         |
| 4.5. Aspectos relevantes de la estrategia de carga . . . . .         | 10        |
| 4.6. Flujo general del pipeline . . . . .                            | 10        |
| <b>5. Aplicación Python para visualización de datos</b>              | <b>11</b> |
| 5.1. Estructura del menú . . . . .                                   | 11        |
| 5.2. Visualización 1: evolución de reviews por año . . . . .         | 11        |
| 5.3. Visualización 2: popularidad de artículos . . . . .             | 11        |
| 5.4. Visualización 3: histograma por nota . . . . .                  | 11        |
| 5.5. Visualización 4: evolución temporal acumulada . . . . .         | 12        |
| 5.6. Visualización 5: histograma de reviews por usuario . . . . .    | 12        |
| 5.7. Visualización 6: nube de palabras . . . . .                     | 12        |
| 5.8. Visualización extra: boxplot por categoría . . . . .            | 13        |

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>6. Resultados de visualización y apoyo con Power BI</b>                     | <b>13</b> |
| 6.1. Evolución temporal por categoría . . . . .                                | 13        |
| 6.2. Histograma global por nota . . . . .                                      | 14        |
| 6.3. Popularidad de artículos . . . . .                                        | 14        |
| <b>7. Aplicación Python con Neo4j</b>                                          | <b>15</b> |
| 7.1. Apartado 4.1: similitudes entre usuarios . . . . .                        | 15        |
| 7.1.1. Idea del apartado . . . . .                                             | 15        |
| 7.1.2. Selección de usuarios top . . . . .                                     | 15        |
| 7.1.3. Cálculo de Pearson . . . . .                                            | 15        |
| 7.2. Apartado 4.2: usuarios y artículos aleatorios de una categoría . . . . .  | 17        |
| 7.3. Apartado 4.3: usuarios que han puntuado más de un tipo de artículo . .    | 18        |
| 7.4. Apartado 4.4: artículos populares y artículos en común entre usuarios . . | 19        |
| 7.5. Visualización automática de los grafos . . . . .                          | 20        |
| <b>8. Inserción de nuevos datasets</b>                                         | <b>20</b> |
| 8.1. Extensión configurable . . . . .                                          | 21        |
| 8.2. Reutilización de la lógica existente . . . . .                            | 21        |
| 8.3. Funcionamiento . . . . .                                                  | 21        |
| <b>9. Relación con Machine Learning y recomendación</b>                        | <b>21</b> |
| 9.1. Separar recomendación simple de modelo de ML . . . . .                    | 21        |
| 9.2. Implementación real del apartado 6.3 . . . . .                            | 22        |
| 9.3. Propuesta de modelo colaborativo . . . . .                                | 22        |
| 9.4. Evolución a factorización matricial . . . . .                             | 23        |
| 9.5. Incorporación de contenido textual . . . . .                              | 23        |
| 9.6. Diagrama conceptual de la propuesta . . . . .                             | 24        |
| <b>10. Dependencias y reproducibilidad</b>                                     | <b>24</b> |
| <b>11. Decisiones de implementación, dificultades y mejoras posibles</b>       | <b>25</b> |
| 11.1. Aspectos positivos del diseño . . . . .                                  | 25        |
| 11.2. Retos técnicos encontrados . . . . .                                     | 25        |
| 11.3. Mejoras futuras . . . . .                                                | 25        |
| <b>12. Conclusiones</b>                                                        | <b>26</b> |

# 1. Introducción

Este proyecto final tiene como objetivo diseñar e implementar una solución de almacenamiento y análisis sobre un conjunto amplio de reseñas de Amazon, combinando tres tecnologías diferenciadas: MySQL para datos estructurados, MongoDB para texto libre y Neo4j para relaciones entre usuarios y artículos.

La motivación principal del trabajo es doble. Por un lado, se busca demostrar que una arquitectura mixta permite representar mejor los datos que un único sistema monolítico. Por otro, se pretende construir una base reutilizable para análisis posteriores como visualizaciones, exploraciones relacionales y un mecanismo de recomendación.

El proyecto se apoya en cuatro datasets de *Amazon Reviews 5-core*, seleccionados por su variedad temática y para permitir estudiar tanto el comportamiento de usuarios como patrones de reviews:

Cuadro 1: Datasets utilizados en el proyecto

| Código       | Categoría                  | Reviews        |
|--------------|----------------------------|----------------|
| 1            | Toys_and_Games_5.json      | 167 597        |
| 2            | Video_Games_5.json         | 231 780        |
| 3            | Digital_Music_5.json       | 64 706         |
| 4            | Musical_Instruments_5.json | 10 261         |
| <b>Total</b> |                            | <b>474 344</b> |

Cada línea de los ficheros JSON representa una review individual con metadatos estructurados y campos de texto libre. Cada registro contiene:

- identificador del usuario (**reviewerID**),
- identificador del artículo (**asin**),
- nombre visible del reviewer,
- utilidad de la review (**helpful**),
- texto completo (**reviewText**),
- resumen (**summary**),
- puntuación (**overall**),
- marca temporal en Unix (**unixReviewTime**),
- fecha legible (**reviewTime**).

## 1.1. Objetivos técnicos

A partir de esos datos, el proyecto persigue los siguientes objetivos:

1. Diseñar un esquema relacional coherente y normalizado para la parte estructurada.
2. Decidir qué campos deben almacenarse en MongoDB y justificar esa separación.

3. Implementar la carga completa desde Python sin leer los ficheros completos en memoria.
4. Programar un script de visualización que consulte directamente las bases de datos.
5. Modelar relaciones en Neo4j para estudiar similitudes y estructuras de consumo.
6. Incorporar una primera lógica de recomendación de artículos no consumidos.

## 1.2. Ficheros entregados

La solución queda organizada en los siguientes ficheros fuente:

- `configuracion.py`: credenciales, nombres de bases de datos y rutas.
- `load_data.py`: creación de la infraestructura y carga principal.
- `menu_visualizacion.py`: menú interactivo de gráficas y recomendación.
- `neo4JProyecto.py`: menú de análisis relacional con Neo4j.
- `insertar_dataset.py`: inserción de nuevos datasets adicionales.
- `apartado_6_2.tex`: desarrollo teórico del modelo de recomendación.
- `requirements.txt`: dependencias necesarias del proyecto.

## 1.3. División del trabajo

La distribución del trabajo siguió el siguiente criterio general:

- **Jorge Ois de Pascual**: modelado relacional, consultas SQL y diseño general del acceso estructurado a los datos.
- **Miguel Pajuelo Gomez**: carga desde JSON, parte MongoDB, visualización en Python e integración analítica.

En la práctica, la integración final obligó a coordinar ambas partes, especialmente en la definición del esquema común y la implementación de Neo4j.

# 2. Planteamiento del problema y decisiones de arquitectura

## 2.1. Por qué una arquitectura mixta

El dataset mezcla dos tipos de información muy distintas:

1. **Datos altamente estructurados**: usuarios, artículos, categorías, notas, fechas y utilidad.
2. **Datos no estructurados**: texto completo y resumen de cada review.

Almacenar todo en MySQL habría simplificado el problema inicial, pero habría introducido dos problemas: primero, el motor relacional no es el más natural para datos no estructurados; segundo, el proyecto pedía explícitamente un uso combinado de base relacional y no relacional. Por lo tanto, la decisión final consistió en separar ambos gestores de la siguiente forma:

Cuadro 2: Reparto de la información entre motores

| Sistema | Campos principales                                                                                                                    | Motivación                                                                                                                      |
|---------|---------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| MySQL   | <code>reviewerID</code> ,<br><code>reviewerName</code> , <code>asin</code> ,<br>categoría, <code>overall</code> , fechas,<br>utilidad | Soporta claves primarias y foráneas, agregaciones, filtros, agrupaciones y joins necesarios para visualización y recomendación. |
| MongoDB | <code>reviewText</code> , <code>summary</code> , junto a <code>reviewerID</code> y <code>asin</code> para enlazar                     | Permite almacenar texto libre de longitud variable sin rigidizar el esquema relacional y facilita el procesamiento textual.     |
| Neo4j   | Usuarios, artículos, tipos y relaciones derivadas                                                                                     | Especializado en representación y consulta de estructuras relacionales complejas.                                               |

## 2.2. Criterio de modelado

El diseño debía cumplir varias restricciones importantes del enunciado:

- no leer un fichero completo en memoria
- no usar `LOAD DATA` de SQL
- parsear la fecha a un tipo específico y no dejarla como string
- tener en cuenta que un mismo usuario puede aparecer en varias categorías.

Eso condiciona directamente la forma de modelar:

- el usuario no puede depender de una categoría concreta
- el artículo debe enlazarse con su categoría
- la review debe descomponerse en metadatos estructurados y texto
- el array `helpful` debe partirse en dos campos atómicos

## 3. Diseño del esquema relacional

### 3.1. Entidades y relaciones

El esquema relacional final se compone de cuatro tablas:

1. **Usuarios**: almacena la identidad lógica del reviewer.

2. **Categorías:** catálogo de categorías tratadas.
3. **Artículos:** referencia cada `asin` y lo enlaza con su categoría.
4. **Reviews\_Metadatos:** guarda la parte estructurada de cada review.

imagenes/diagrama\_mysql.png

Figura 1: Esquema relacional implementado en MySQL

### 3.2. Justificación detallada

**Usuarios** El par `reviewerID-reviewerName` se almacena en una tabla independiente porque el mismo usuario puede escribir reviews en varias categorías y sobre múltiples artículos. Esta decisión nos evitó redundancia y nos facilitó consultas por usuario.

**Categorías** Aunque el número de categorías trabajadas es pequeño, las modelamos en una tabla propia para evitar repetir cadenas literales y permitir que el sistema pueda ampliarse fácilmente con nuevos datasets mediante inserción de nuevas filas.

**Artículos** Cada artículo se identifica por `asin`. En el diseño adoptado, el artículo referencia una categoría concreta. Esta decisión simplifica las consultas de agregación por tipo de producto y hace posible recorrer la cadena:

Review  $\rightarrow$  Artículo  $\rightarrow$  Categoría

**Reviews\_Metadatos** La review es la entidad transaccional del sistema: representa el hecho de que un usuario ha puntuado un artículo en un momento determinado. Todos los campos numéricos o temporales relevantes para análisis y gráficos se concentran aquí.

### 3.3. Representación de la parte no estructurada en MongoDB

La colección principal de MongoDB almacena documentos de la siguiente forma:

```

1 {
2     "reviewerID": "...",
3     "asin": "...",
4     "reviewText": "...",
5     "summary": "..."
6 }

```

No se replica la categoría en MongoDB porque puede recuperarse desde MySQL a través del `asin`. Esta elección minimiza duplicidad y mantiene MongoDB centrado exclusivamente en el contenido textual.

## 4. Implementación de la carga de datos

### 4.1. Archivo de configuración

Una condición importante del proyecto es que se pueda cambiar fácilmente las rutas de los datasets y las credenciales sin tocar la lógica principal. Esa centralización se resuelve en `configuracion.py`. El archivo separa tres bloques: configuración MySQL, configuración MongoDB y rutas de entrada.

```

1 MYSQL_CONFIG = {
2     "user": "proyecto",
3     "password": "***",
4     "host": "127.0.0.1",
5     "database": "amazon_reviews_db"
6 }
7
8 MONGO_CONFIG = {
9     "uri": "mongodb://localhost:27017/",
10    "db_name": "amazon_reviews_db",
11    "collection_name": "reviews_textos"
12 }
13
14 DATA_FILES = {
15     "Digital_Music": "datos/Digital_Music_5.json",
16     "Musical_Instruments": "datos/Musical_Instruments_5.json",
17     "Toys_and_Games": "datos/Toys_and_Games_5.json",
18     "Video_Games": "datos/Video_Games_5.json"
19 }

```

Esta organización aporta tres ventajas claras:

- hace la carga reproducible en otra máquina;
- evita “hardcodear” rutas por todo el proyecto;
- permite reutilizar la misma configuración desde varios scripts.

### 4.2. Conexión y creación de la infraestructura

El fichero `load_data.py` se encarga de abrir las conexiones y crear las tablas si no existen. El diseño de la función `connect_db()` crea primero la conexión MySQL sin

seleccionar base de datos, lo que permite ejecutar `CREATE DATABASE IF NOT EXISTS` antes de trabajar sobre el esquema.

```
1 def parse_date(date_str):
2     """Convierte '07 9, 2012' a objeto datetime para MySQL"""
3     return datetime.strptime(date_str, '%m %d, %Y')
4
5 def connect_db():
6     config_sin_db = config.MYSQL_CONFIG.copy()
7     config_sin_db.pop("database")
8
9     db_mysql = pymysql.connect(**config_sin_db)
10    cursor = db_mysql.cursor()
11
12    client_mongo = MongoClient(config.MONGO_CONFIG['uri'])
13    db_mongo = client_mongo[config.MONGO_CONFIG['db_name']]
14    coll_mongo = db_mongo[config.MONGO_CONFIG['collection_name']]
15
16    return db_mysql, cursor, coll_mongo
```

La fecha textual `reviewTime` se transforma a tipo fecha con `datetime.strptime`. Esta conversión es una parte crítica solicitada por el enunciado, porque evita almacenar fechas como cadenas sin semántica temporal.

### 4.3. Creación de tablas

La estructura de MySQL se construye desde Python. La parte central del script es la siguiente:

```
1 cursor.execute("""
2     CREATE TABLE IF NOT EXISTS Usuarios (
3         reviewerID    VARCHAR(50) PRIMARY KEY,
4         reviewerName  VARCHAR(50)
5     )
6 """)
7
8 cursor.execute("""
9     CREATE TABLE IF NOT EXISTS Categorias (
10        id_categoria    INT AUTO_INCREMENT PRIMARY KEY,
11        nombre_categoria VARCHAR(50) UNIQUE
12    )
13 """)
14
15 cursor.execute("""
16     CREATE TABLE IF NOT EXISTS Articulos (
17         asin           VARCHAR(20) PRIMARY KEY,
18         id_categoria   INT,
19         FOREIGN KEY (id_categoria) REFERENCES
20         Categorias(id_categoria)
21     )
22 """)
```

```

23 cursor.execute("""
24     CREATE TABLE IF NOT EXISTS Reviews_Metadatos (
25         id_reviews          INT AUTO_INCREMENT PRIMARY KEY,
26         reviewerID          VARCHAR(50),
27         asin                 VARCHAR(20),
28         overall              REAL,
29         unixReviewTime       INT,
30         reviewTime           DATE,
31         helpful_votes        INT,
32         helpful_total        INT,
33         FOREIGN KEY (reviewerID) REFERENCES Usuarios(reviewerID),
34         FOREIGN KEY (asin) REFERENCES Articulos(asin)
35     )
36 """)

```

A nivel de diseño, esta parte garantiza tres propiedades:

1. integridad referencial entre tablas;
2. trazabilidad desde una review hasta su usuario y su artículo;
3. soporte directo para consultas analíticas por usuario, artículo, categoría, nota o tiempo.

#### 4.4. Carga incremental línea a línea

El requisito más delicado de la práctica era no cargar los JSON completos en memoria. La solución que hemos planteado es recorrer cada fichero línea a línea, parseando cada review y distribuyendo la información entre MySQL y MongoDB.

```

1 with open(file_path, "r", encoding="utf-8") as f:
2     for line in f:
3         data = json.loads(line)
4         fecha_sql =
5             parse_date(data["reviewTime"]).strftime("%Y-%m-%d")
6
7         cursor.execute(
8             "INSERT IGNORE INTO Usuarios (reviewerID,
9             reviewerName) VALUES (%s, %s)",
10            (data['reviewerID'], data.get('reviewerName',
11            'Unknown'))
12        )
13
14        cursor.execute(
15            "INSERT IGNORE INTO Articulos (asin, id_categoria)
16            VALUES (%s, %s)",
17            (data['asin'], id_cat)
18        )
19
20        cursor.execute(
21            """INSERT INTO Reviews_Metadatos
22            (reviewerID, asin, overall, unixReviewTime,
23            reviewTime,

```

```

19         helpful_votes, helpful_total)
20         VALUES (%s, %s, %s, %s, %s, %s, %s)""" ,
21         (data['reviewerID'], data['asin'], data['overall'],
22         data['unixReviewTime'], fecha_sql,
23         data['helpful'][0], data['helpful'][1])
24     )
25
26     mongo_buffer.append({
27         "reviewerID": data['reviewerID'],
28         "asin": data['asin'],
29         "reviewText": data.get('reviewText', ''),
30         "summary": data.get('summary', '')
31     })
32
33     if len(mongo_buffer) >= 500:
34         coll_mongo.insert_many(mongo_buffer)
35         mongo_buffer = []
36         db_mysql.commit()

```

#### 4.5. Aspectos relevantes de la estrategia de carga

- **Inserción incremental:** cada línea se procesa de forma independiente.
- **Evitar duplicados:** INSERT IGNORE impide insertar varias veces usuarios o artículos ya existentes.
- **Partición del campo helpful:** el array original se separa en helpful\_votes y helpful\_total.
- **Buffer para MongoDB:** los textos se insertan en bloques de 500 documentos para reducir overhead.
- **Commit por lotes:** se reduce el coste de confirmar transacciones demasiado pequeñas.

#### 4.6. Flujo general del pipeline

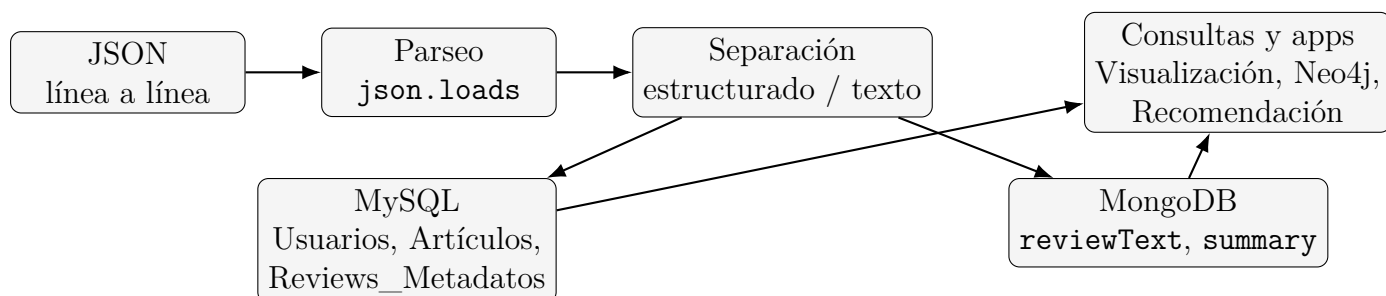


Figura 2: Pipeline general del proyecto

## 5. Aplicación Python para visualización de datos

### 5.1. Estructura del menú

El fichero `menu_visualizacion.py` implementa una aplicación de consola que mantiene un bucle interactivo hasta que el usuario selecciona la opción de salida. El menú final integra todos apartados de visualización del enunciado así como la funcionalidad de recomendación pedida al final del proyecto.

```
1 print("1. Evolución de reviews por años")
2 print("2. Popularidad de artículos")
3 print("3. Histograma por nota (overall)")
4 print("4. Evolución temporal acumulada")
5 print("5. Histograma de reviews por usuario")
6 print("6. Nube de palabras (Summary)")
7 print("7. Gráfico extra (Boxplot por categoría)")
8 print("8. Funcionalidad apartado 6.3: recomendar 10 artículos no
    consumidos")
9 print("9. Salir")
```

A nivel funcional, el programa se conecta simultáneamente a MySQL y MongoDB, y según la opción elegida consulta una u otra base de datos, o ambas.

### 5.2. Visualización 1: evolución de reviews por año

La primera gráfica agrupa las reviews por año utilizando `YEAR(reviewTime)`. Si el usuario selecciona una categoría concreta, el sistema realiza un `JOIN` con `Articulos` y `Categorias`. En caso contrario, agrega sobre el total.

```
1 SELECT YEAR(reviewTime) as anio, COUNT(*) as total
2 FROM Reviews_Metadatos
3 GROUP BY anio
4 ORDER BY anio
```

Este análisis permite detectar la concentración temporal de las reseñas y comprobar en qué periodos crece más el volumen de información disponible.

### 5.3. Visualización 2: popularidad de artículos

La popularidad se define como el número de reviews asociadas a un artículo. La implementación genera una curva ordenando los artículos por número de valoraciones.

```
1 SELECT asin, COUNT(*) as num_reviews
2 FROM Reviews_Metadatos
3 GROUP BY asin
4 ORDER BY num_reviews DESC
```

El resultado esperado es una distribución desbalanceada: unos pocos artículos muy populares y una gran masa de artículos con pocas reseñas.

### 5.4. Visualización 3: histograma por nota

Para el histograma de puntuaciones se agrega el campo `overall`:

```

1 SELECT overall, COUNT(*)
2 FROM Reviews_Metadatos
3 GROUP BY overall
4 ORDER BY overall

```

El comportamiento observado en la práctica está claramente sesgado hacia puntuaciones altas, especialmente 4 y 5. Esto es importante porque condiciona cualquier modelo de recomendación futuro: la variable objetivo tiene una distribución poco equilibrada.

## 5.5. Visualización 4: evolución temporal acumulada

La implementación del menú traza la evolución acumulada ordenando los `unixReviewTime` y construyendo un vector acumulativo.

```

1 query = "SELECT unixReviewTime FROM Reviews_Metadatos ORDER BY
          unixReviewTime"
2 timestamps = [r[0] for r in resultados]
3 acumulado = list(range(1, len(timestamps) + 1))

```

Aunque esta versión es global y no desagrega por categoría en la aplicación principal, sí permite observar el crecimiento total del corpus disponible a lo largo del tiempo.

## 5.6. Visualización 5: histograma de reviews por usuario

Esta visualización cuenta cuántas reviews ha escrito cada usuario y después construye un histograma sobre esa distribución. Su interés analítico es claro: muestra si la actividad del sistema está repartida homogéneamente o concentrada en una minoría de usuarios especialmente activos.

## 5.7. Visualización 6: nube de palabras

Para la nube de palabras se combinan ambos ges. Primero se consulta MySQL para obtener los `asin` de la categoría seleccionada; después se buscan en MongoDB los resúmenes asociados. El texto se limpia con expresiones regulares y se genera la nube usando la librería `wordcloud`.

```

1 busqueda = coll_mongo.find(
2     {"asin": {"$in": asins}},
3     {"summary": 1, "_id": 0}
4 )
5
6 texto_completo = " ".join(textos).lower()
7 texto_completo = re.sub(r"[^a-zA-Z\s]", " ", texto_completo)
8 texto_completo = re.sub(r"\s+", " ", texto_completo).strip()

```

Esta parte es especialmente representativa del enfoque híbrido: MySQL filtra por categoría y MongoDB aporta el contenido textual a procesar.

## 5.8. Visualización extra: boxplot por categoría

Como gráfica adicional se implementa un boxplot de `overall` por categoría. La consulta se apoya en joins entre las tablas estructuradas y usa `pandas` únicamente para facilitar el dibujo con `seaborn`.

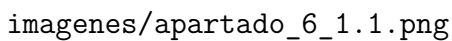
Escogimos boxplot por su potencial doble funcionalidad:

- comparar visualmente dispersión y mediana entre categorías;
- detectar posibles outliers o diferencias sistemáticas entre dominios.

## 6. Resultados de visualización y apoyo con Power BI

Además de las gráficas generadas desde Python, el proyecto incluye tres figuras elaboradas con PowerBI para reforzar la parte visual del informe. Estas gráficas permiten presentar de forma más limpia algunos patrones globales del dataset.

### 6.1. Evolución temporal por categoría



imagenes/apartado\_6\_1.1.png

Figura 3: Visualización externa de la evolución temporal por categoría

La gráfica permite comparar categorías entre sí. Destaca el predominio del volumen asociado a `Video_Games` y, en segundo plano, `Toys_and_Games`. Las categorías `Digital_Music` y `Musical_Instruments` presentan una escala mucho menor.

## 6.2. Histograma global por nota

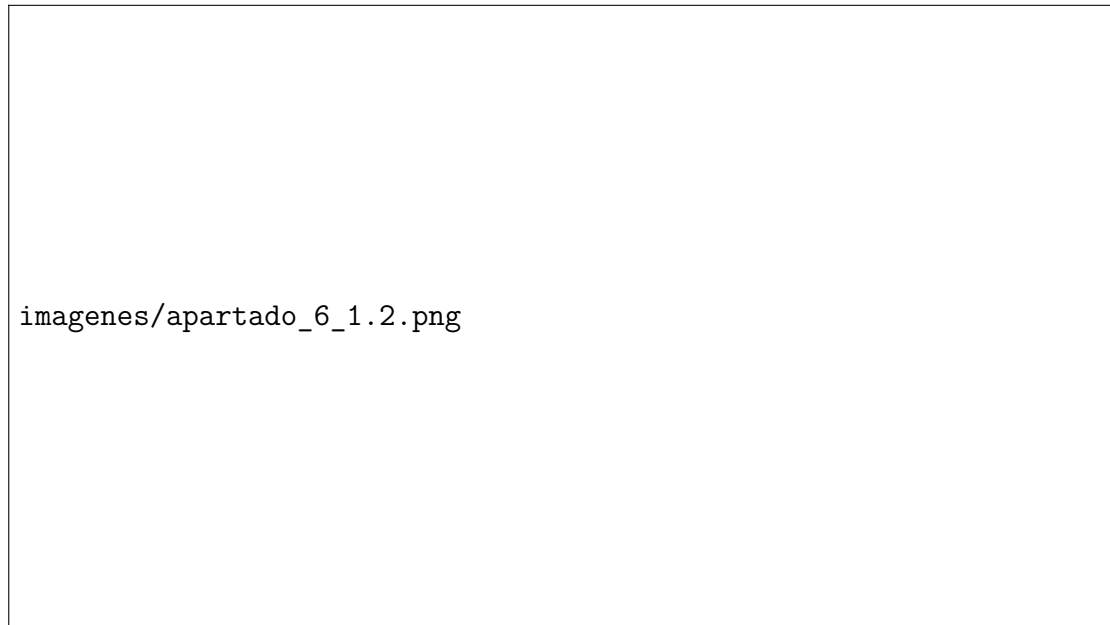


Figura 4: Distribución global de reviews por nota

La distribución refuerza la hipótesis observada al principio del informe: las reviews de Amazon no son simétricas. La categoría de cinco estrellas domina claramente el histograma, seguida por cuatro y tres, mientras que uno y dos aparecen con mucha menos frecuencia.

## 6.3. Popularidad de artículos

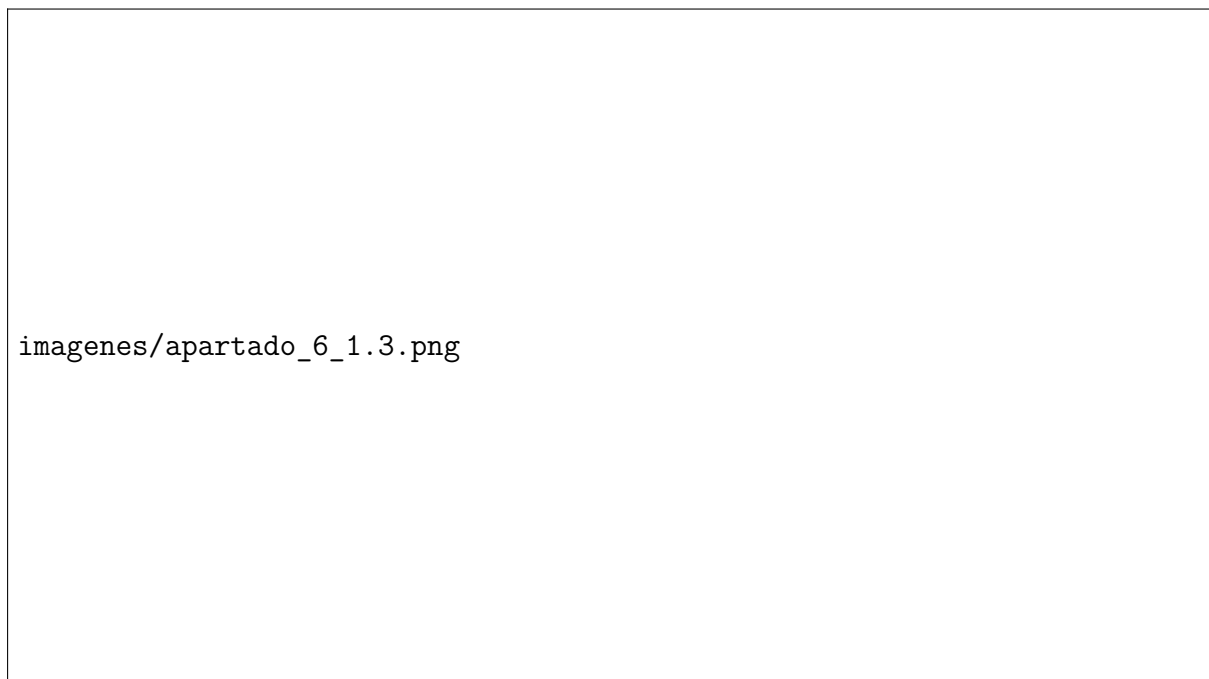


Figura 5: Popularidad de artículos ordenados por volumen de reseñas

Esta visualización ilustra con claridad la *long tail* del catálogo: unos pocos *asin* concentran gran parte de las interacciones, mientras que la mayoría queda muy por debajo.

## 7. Aplicación Python con Neo4j

El fichero `neo4JProyecto.py` extiende el proyecto hacia un análisis relacional. En lugar de pensar solo en tablas, en este apartado hemos desarrollado grafos a partir de las interacciones almacenadas en MySQL. El script define parámetros fácilmente modificables como se solicitaba en

```
1 TOP_USUARIOS_SIMILITUD = 30
2 PRIMEROS_USUARIOS = 400
3 TOP_ARTICULOS_POPULARES = 5
4 MAX_REVIEWS_ARTICULO_POPULAR = 40
```

### 7.1. Apartado 4.1: similitudes entre usuarios

#### 7.1.1. Idea del apartado

Se seleccionan los 30 usuarios con más reviews, se recuperan sus artículos puntuados y se calcula la correlación de Pearson entre cada par. Solo se almacenan similitudes entre usuarios que comparten al menos un artículo.

#### 7.1.2. Selección de usuarios top

```
1 SELECT u.reviewerID, u.reviewerName,
2         COUNT(*) AS total_reviews,
3         AVG(r.overall) AS media_reviews
4 FROM Usuarios u
5 JOIN Reviews_Metadatos r ON u.reviewerID = r.reviewerID
6 GROUP BY u.reviewerID, u.reviewerName
7 ORDER BY total_reviews DESC, u.reviewerID ASC
8 LIMIT %s
```

#### 7.1.3. Cálculo de Pearson

```
1 def calcular_pearson(ratings_u, ratings_v, media_u, media_v):
2     articulos_comunes = set(ratings_u.keys()) &
3     set(ratings_v.keys())
4
5     if not articulos_comunes:
6         return None, 0
7
8     numerador = 0
9     suma_u = 0
10    suma_v = 0
11
12    for asin in articulos_comunes:
13        dif_u = ratings_u[asin] - media_u
```

```

13     dif_v = ratings_v[asin] - media_v
14     numerador += dif_u * dif_v
15     suma_u += dif_u ** 2
16     suma_v += dif_v ** 2
17
18     denominador = sqrt(suma_u) * sqrt(suma_v)
19
20     if denominador == 0:
21         return 0.0, len(articulos_comunes)
22
23     return numerador / denominador, len(articulos_comunes)

```

El resultado se carga en Neo4j como relaciones `SIMILAR_A` bidireccionales con dos propiedades:

- `pearson`: intensidad de similitud,
- `common_items`: número de artículos puntuados en común.

imagenes/apartado\_4\_1.png

Figura 6: Grafo del apartado 4.1: similitudes entre los usuarios más activos

El script incluye además una consulta final para mostrar por pantalla el usuario con mayor número de vecinos dentro del grafo generado.

## 7.2. Apartado 4.2: usuarios y artículos aleatorios de una categoría

En esta parte se pide al usuario una categoría y un número  $n$  de artículos. El sistema selecciona artículos aleatorios de esa categoría y recupera todas las reviews asociadas.

```
1 SELECT a.asin, c.nombre_categoria
2 FROM Articulos a
3 JOIN Categorias c ON a.id_categoria = c.id_categoria
4 WHERE c.nombre_categoria = %s
5 ORDER BY RAND()
6 LIMIT %s
```

Posteriormente se cargan tres tipos de información en Neo4j:

1. nodos `Articulo`,
2. nodos `Usuario`,
3. relaciones `VALORO` con propiedades `rating`, `unixReviewTime` y `reviewTime`.

imagenes/apartado\_4\_2.png

Figura 7: Grafo del apartado 4.2: artículos aleatorios y usuarios que los valoraron

Este apartado es útil porque permite ver qué usuarios convergen alrededor de un conjunto reducido de artículos concretos.

### 7.3. Apartado 4.3: usuarios que han puntuado más de un tipo de artículo

La lógica de este bloque es distinta: ya no interesan artículos concretos, sino diversidad temática. En el script hemos hecho que seleccione los primeros 400 usuarios ordenados por nombre, que calcule para cada uno cuántos artículos ha puntuado en cada categoría y conserve solo aquellos que han puntuado artículos de más de un tipo.

```
1 SELECT u.reviewerID, u.reviewerName, c.nombre_categoria,
2        COUNT(DISTINCT r.asin) AS num_articulos
3 FROM Usuarios u
4 JOIN Reviews_Metadatos r ON u.reviewerID = r.reviewerID
5 JOIN Articulos a ON r.asin = a.asin
6 JOIN Categorias c ON a.id_categoria = c.id_categoria
7 WHERE u.reviewerID IN (...)
8 GROUP BY u.reviewerID, u.reviewerName, c.nombre_categoria
```

En Neo4j, este resultado se representa con:

- nodos Usuario,
- nodos TipoArticulo,
- relaciones HA\_PUNTUADO\_TIPO con propiedad num\_articulos.

imagenes/apartado\_4\_3.png

Figura 8: Grafo del apartado 4.3: usuarios multicategoría y tipos consumidos

El valor analítico de esta representación es claro: permite detectar usuarios generalistas frente a usuarios muy especializados.

## 7.4. Apartado 4.4: artículos populares y artículos en común entre usuarios

En este caso seleccionamos los 5 artículos más populares que tengan menos de 40 reviews. La restricción del máximo evita que el grafo se vuelva inmanejable por exceso de aristas.

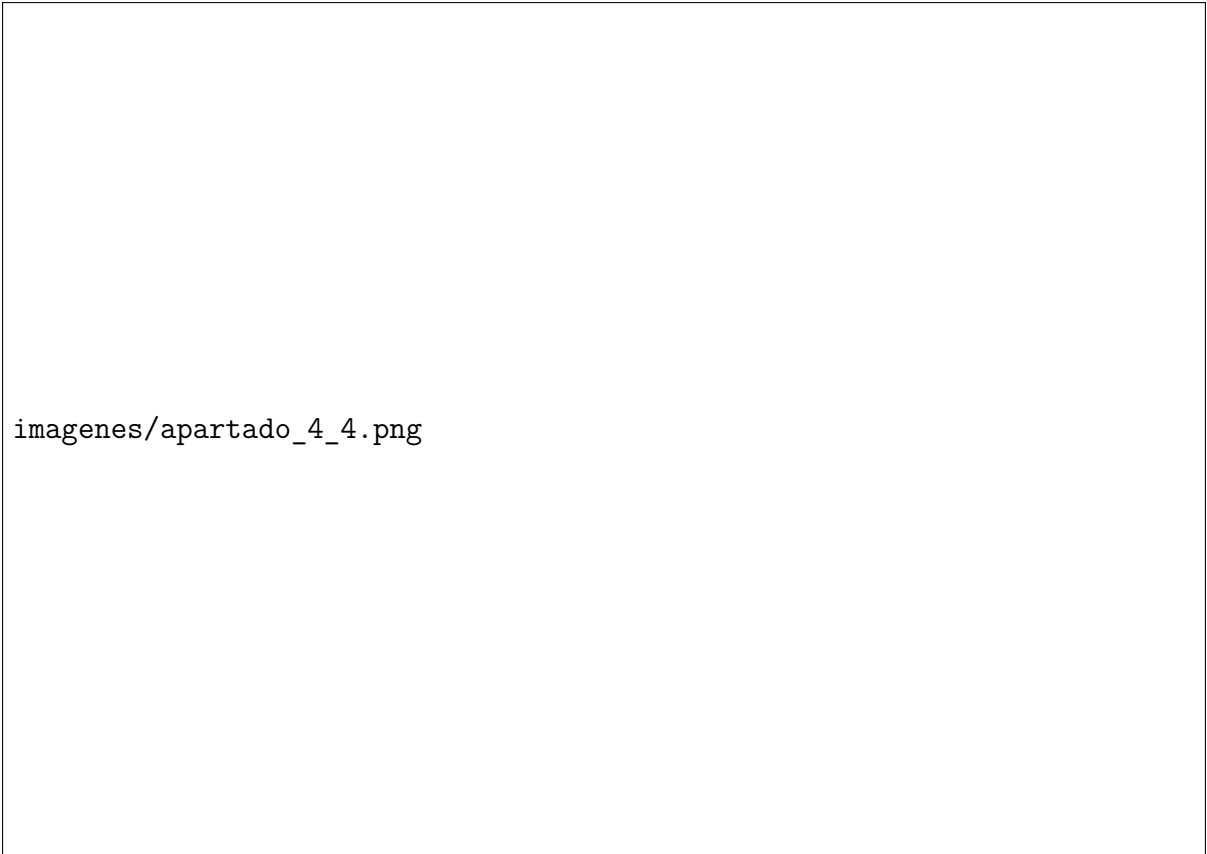
```
1 SELECT r.asin, c.nombre_categoria, COUNT(*) AS num_reviews
2 FROM Reviews_Metadatos r
3 JOIN Articulos a ON r.asin = a.asin
4 JOIN Categorias c ON a.id_categoria = c.id_categoria
5 GROUP BY r.asin, c.nombre_categoria
6 HAVING COUNT(*) < %s
7 ORDER BY num_reviews DESC, r.asin ASC
8 LIMIT %s
```

Después se obtienen todos los usuarios implicados y se calcula cuántos artículos en común ha puntuado cada par:

```
1 SELECT r1.reviewerID AS user1,
2        r2.reviewerID AS user2,
3        COUNT(DISTINCT r1.asin) AS common_items
4 FROM Reviews_Metadatos r1
5 JOIN Reviews_Metadatos r2
6     ON r1.asin = r2.asin
7     AND r1.reviewerID < r2.reviewerID
8 WHERE r1.reviewerID IN (...)
9     AND r2.reviewerID IN (...)
10 GROUP BY r1.reviewerID, r2.reviewerID
11 HAVING common_items > 0
```

El grafo final contiene:

- artículos populares como nodos,
- usuarios que los han puntuado,
- relaciones VALORO,
- relaciones ARTICULOS\_EN\_COMUN entre pares de usuarios.



imagenes/apartado\_4\_4.png

Figura 9: Grafo del apartado 4.4: artículos populares y usuarios con artículos en común

## 7.5. Visualización automática de los grafos

Un aspecto especialmente interesante del proyecto es que las capturas anteriores no proceden directamente del navegador de Neo4j, sino de una visualización programática construida con `networkx` y `matplotlib`. El script lee los nodos y relaciones actuales desde Neo4j y genera una imagen PNG.

```
1 MATCH (n)
2 RETURN elementId(n) AS id, labels(n) AS labels, properties(n) AS
   props
3
4 MATCH (n)-[r]->(m)
5 RETURN elementId(n) AS origen,
6         elementId(m) AS destino,
7         type(r) AS tipo,
8         properties(r) AS props
```

A partir de esa consulta genérica, se asignan etiquetas y colores según el tipo de nodo y se dibuja el grafo con un `spring_layout` característico de `networkx`.

## 8. Inserción de nuevos datasets

El enunciado pedía diseñar un mecanismo para incorporar un dataset adicional distinto de los cuatro principales. Esa necesidad se resuelve en `insertar_dataset.py`.

## 8.1. Extensión configurable

El script utiliza un nuevo diccionario de rutas en el archivo de configuración:

```
1 EXTRA_DATA_FILES = {  
2     "Instant_Video": "...",  
3     "Phones_and_Accessories": "...",  
4     "Clothing_Shoes_and_Jewelry": "...",  
5     "Grocery_and_Food": "...",  
6     "Office_Products": "...",  
7     "Pet_Supplies": "...",  
8     "Sports_and_Outdoors": "..."  
9 }
```

## 8.2. Reutilización de la lógica existente

En lugar de duplicar la lógica de carga, el script reutiliza componentes del cargador principal:

```
1 from load_data import connect_db, parse_date
```

Eso tiene dos ventajas:

1. consistencia en el tratamiento de fechas y conexiones;
2. menor mantenimiento, porque un cambio en la lógica principal se hereda más fácilmente.

## 8.3. Funcionamiento

El usuario puede seleccionar el dataset extra por nombre o por índice. A continuación, el script:

- inserta la nueva categoría si no existe,
- obtiene su `id_categoria`,
- recorre el nuevo fichero línea a línea,
- vuelve a insertar usuarios y artículos con `INSERT IGNORE`,
- añade metadatos a MySQL y texto a MongoDB.

Este diseño respeta la idea original de *extensibilidad*: el sistema no queda cerrado a los cuatro datasets iniciales.

# 9. Relación con Machine Learning y recomendación

## 9.1. Separar recomendación simple de modelo de ML

En el proyecto aparecen dos niveles distintos de recomendación:

1. una **implementación directa y funcional** basada en popularidad para el apartado 6.3;
2. una **propuesta metodológica** más ambiciosa, desarrollada en el siguiente apartado del informe, para un recomendador de tipo colaborativo / híbrido.

Es importante no confundir ambas capas. La primera ya está programada y ejecutable; la segunda es una propuesta fundamentada para una evolución futura del sistema.

## 9.2. Implementación real del apartado 6.3

La funcionalidad ya implementada solicita un `reviewerID` y una categoría, comprueba que ambos existan y devuelve los 10 artículos más populares de esa categoría que el usuario todavía no ha consumido.

```

1 SELECT a.asin, COUNT(r.asin) AS num_reviews
2 FROM Articulos a
3 JOIN Categorias c ON a.id_categoria = c.id_categoria
4 JOIN Reviews_Metadatos r ON a.asin = r.asin
5 WHERE c.nombre_categoria = %s
6     AND a.asin NOT IN (
7         SELECT asin
8         FROM Reviews_Metadatos
9         WHERE reviewerID = %s
10    )
11 GROUP BY a.asin
12 ORDER BY num_reviews DESC, a.asin ASC
13 LIMIT 10

```

Este mecanismo no usa todavía aprendizaje automático. Es una heurística basada en popularidad condicionada por historial de consumo. Sin embargo, tiene valor práctico inmediato:

- evita recomendar artículos ya vistos por el usuario;
- restringe la recomendación a un tipo de producto concreto;
- devuelve una lista razonable incluso sin entrenar ningún modelo.

## 9.3. Propuesta de modelo colaborativo

La idea que planteamos para resolver el enunciado consiste en construir una matriz usuario-artículo a partir de las valoraciones `overall` y estimar afinidades entre usuarios.

La medida propuesta se basa en la previamente vista correlación de Pearson:

$$PC(u, v) = \frac{\sum_{i \in I_{uv}} (r_{u,i} - \bar{r}_u)(r_{v,i} - \bar{r}_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_{i \in I_{uv}} (r_{v,i} - \bar{r}_v)^2}}$$

Esta elección es coherente con la parte de Neo4j ya implementada. De hecho, el apartado 4.1 puede interpretarse como un primer paso exploratorio hacia un recomendador basado en vecinos.

## 9.4. Evolución a factorización matricial

A mayores, proponemos sustituir el modelo basado en vecinos por una factorización matricial. El objetivo sería representar usuarios y artículos por vectores latentes  $p_u$  y  $q_i$ , y estimar la afinidad mediante:

$$\hat{r}_{u,i} = p_u^T q_i$$

Esta aproximación tiene varias ventajas sobre el filtrado colaborativo puro:

- se comporta mejor en matrices dispersas;
- generaliza mejor que las reglas locales basadas en vecinos;
- permite capturar factores implícitos de gusto o estilo de consumo.

## 9.5. Incorporación de contenido textual

MongoDB no se ha usado solo por obligación del enunciado; también es una vía real para enriquecer el sistema de recomendaciones del modelo. El texto de `reviewText` y `summary` podría vectorizarse con TF-IDF o embeddings y combinarse con la señal colaborativa. Eso daría lugar a un recomendador híbrido de la forma:

$$Score(u, i) = \alpha \cdot Score_{colaborativo}(u, i) + (1 - \alpha) \cdot Score_{contenido}(u, i)$$

donde  $\alpha$  controla el equilibrio entre historial de consumo y similitud semántica del contenido.

## 9.6. Diagrama conceptual de la propuesta

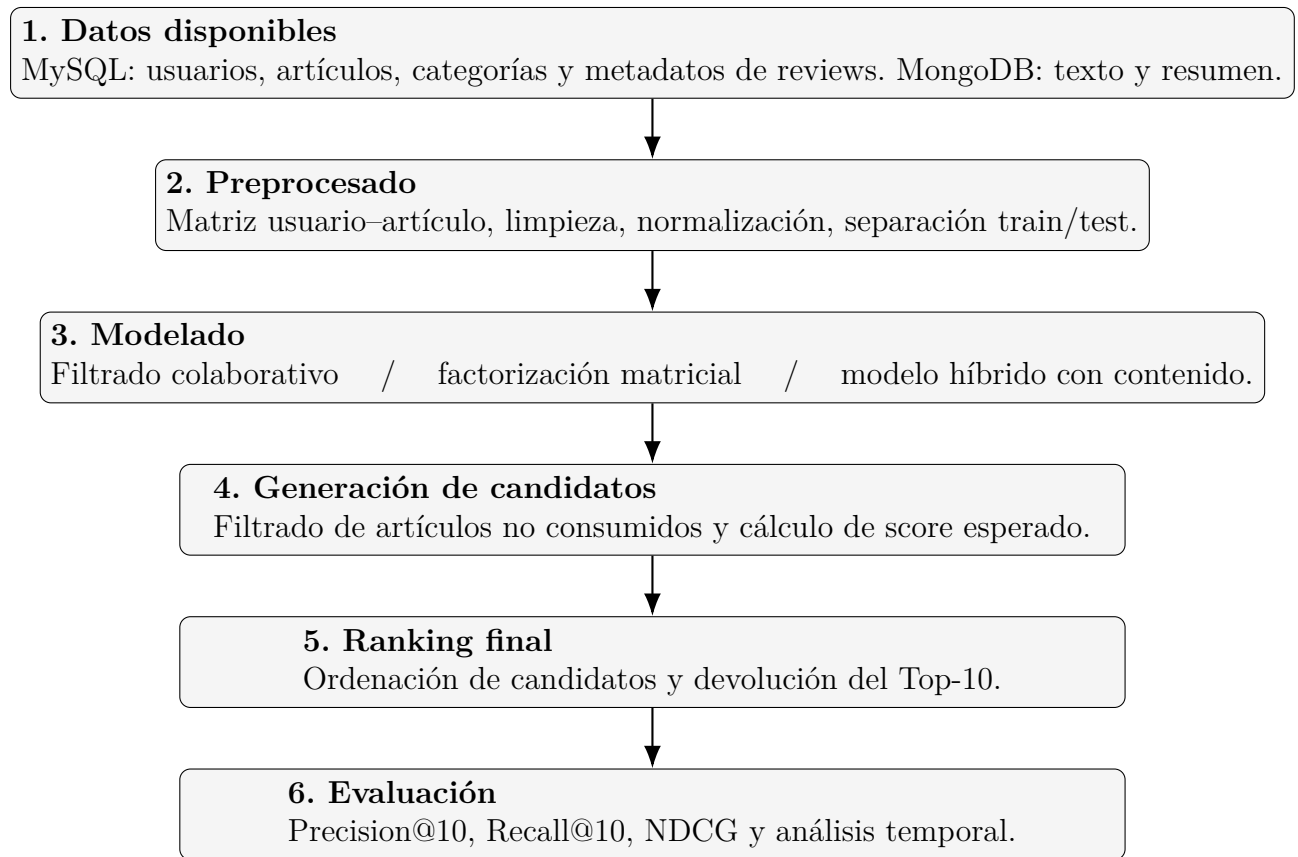


Figura 10: Propuesta metodológica de evolución hacia un sistema de recomendación

## 10. Dependencias y reproducibilidad

El proyecto incluye un `requirements.txt` con las librerías necesarias:

```
1 pymysql>=1.1.0
2 pymongo>=4.6.0
3 matplotlib>=3.8.0
4 seaborn>=0.13.0
5 wordcloud>=1.9.0
6 neo4j>=5.0.0
7 networkx>=3.0
8 pandas>=2.1.0,<3.0.0
```

Estas dependencias cubren exactamente las piezas funcionales del proyecto:

- conexión con MySQL,
- conexión con MongoDB,
- gráficos estáticos,
- nube de palabras,
- conexión y renderizado de grafos,

- apoyo tabular con `pandas`.

Las versiones específicas de las librerías fueron recomendadas por ChatGPT cuando le solicitamos que maximizase la compatibilidad entre librerías y respecto al proyecto.

## 11. Decisiones de implementación, dificultades y mejoras posibles

### 11.1. Aspectos positivos del diseño

El proyecto resuelve bien varios aspectos difíciles del enunciado:

- **Cumplimiento de la carga incremental:** no se lee el fichero completo de una, sino que lo lee línea por línea.
- **Separación razonada de SGBBDD:** cada tecnología se usa para lo que mejor resuelve.
- **Extensibilidad:** es sencillo añadir datasets extra.
- **Parámetros configurables:** tanto rutas como cantidades clave están centralizadas.
- **Reutilización:** varias piezas de lógica se reaprovechan entre scripts.

### 11.2. Retos técnicos encontrados

Durante un proyecto de este tipo aparecen dificultades inevitables:

1. **Volumen de datos:** incluso con datasets 5-core, el total de reviews es suficientemente grande como para obligar a procesar de forma incremental y la carga de datos no es inmediata.
2. **Integridad entre SGDBBDD:** al dividir la información entre MySQL y MongoDB hay que mantener un criterio de enlace claro, en este caso `asin` y `reviewerID`.
3. **Diseño relacional correcto:** un usuario puede aparecer en varias categorías, por lo que no se puede modelar localmente dentro de cada dataset.
4. **Representación visual:** los grafos densos son informativos pero pueden volverse difíciles de leer, por lo que fue necesario limitar tamaños y filtrar escenarios concretos.

### 11.3. Mejoras futuras

Aunque la base del proyecto es sólida, existen mejoras razonables:

- crear índices adicionales sobre campos muy consultados, como `reviewerID`, `asin` y `unixReviewTime`;

- ampliar los filtros por categoría en algunas visualizaciones para homogeneizar completamente el comportamiento del menú;
- añadir validación más robusta de errores y rollback transaccional fino;
- externalizar también la configuración de Neo4j al archivo de configuración general;
- materializar un sistema de recomendación entrenado y evaluado con métricas formales.

## 12. Conclusiones

El proyecto demuestra que una arquitectura híbrida no solo es viable, sino útil, cuando el dato combina estructura tabular, texto libre y relaciones complejas. MySQL ha servido como columna vertebral para el análisis estructurado. MongoDB ha permitido separar de manera limpia el contenido textual. Neo4j ha añadido una dimensión relacional que no resulta natural en el modelo tabular clásico.

Desde el punto de vista académico, el trabajo cubre el ciclo completo de un pequeño sistema de datos:

1. Modelado
2. Ingestión
3. Persistencia
4. Consulta,
5. Visualización
6. Análisis relacional
7. Primera capa de recomendación

Además, deja preparada una base para evolucionar hacia un recomendador más avanzado. Esta continuidad es una de las virtudes del proyecto: no se queda en una práctica aislada, sino que establece un modelo sobre el que se pueden construir componentes de Machine Learning, análisis semántico y sistemas híbridos más sofisticados.

En conjunto, el resultado final no es solo una colección de scripts, sino un sistema integrado que transforma datos crudos de reviews en una infraestructura analítica reutilizable y extensible.